DIGITALHOUSE CLASE EN VIVO 4

ARCA | AGENCIA DE RECAUDACIÓN Y CONTROL ADUANERO

Orquestación y gobierno de datos

Clase en vivo 4 · 120 minutos

Programa: Arquitectura de Datos, Gobierno y Orquestación — ARCA

Escaneá el QR para registrar tu asistencia

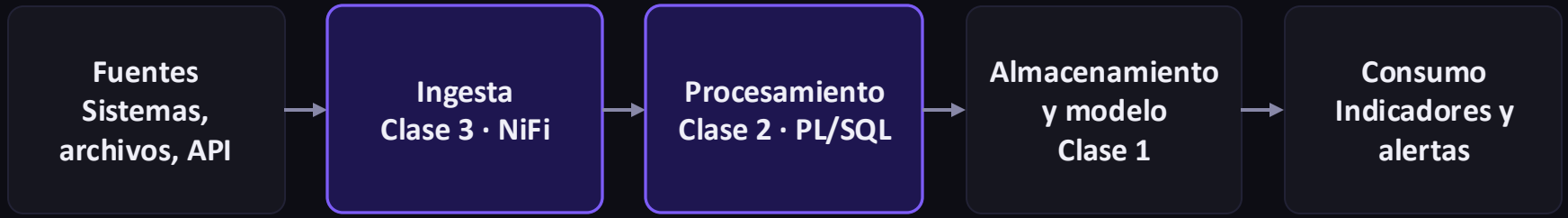
Me presento...

1. Directora General en The eLabs: transformación digital e integración de IA.
2. Host del podcast Hack The Future: estrategia y liderazgo, no solo algoritmos.
3. Docente en la Maestría en Ciencia de Datos (Univ. Austral), ITBA y Di Tella.



Apasionada por aplicar IA para mejorar la productividad y las decisiones.

Revisión de procesos



¿Quién coordina cuándo corre cada cosa y qué hace si falla?
→ Orquestación

¿Quién responde por el dato y cómo sabemos que está bien?
→ Gobierno y calidad

Qué vamos a lograr hoy

PROPÓSITO

Integrar una DAG con contratos de calidad, responsables y criterios de recuperación.

RESULTADO OBSERVABLE

Una DAG comentada, una matriz RACI breve y controles bloqueantes y no bloqueantes.

Siete equipos, siete desafíos: cada uno integra todos los temas de hoy.

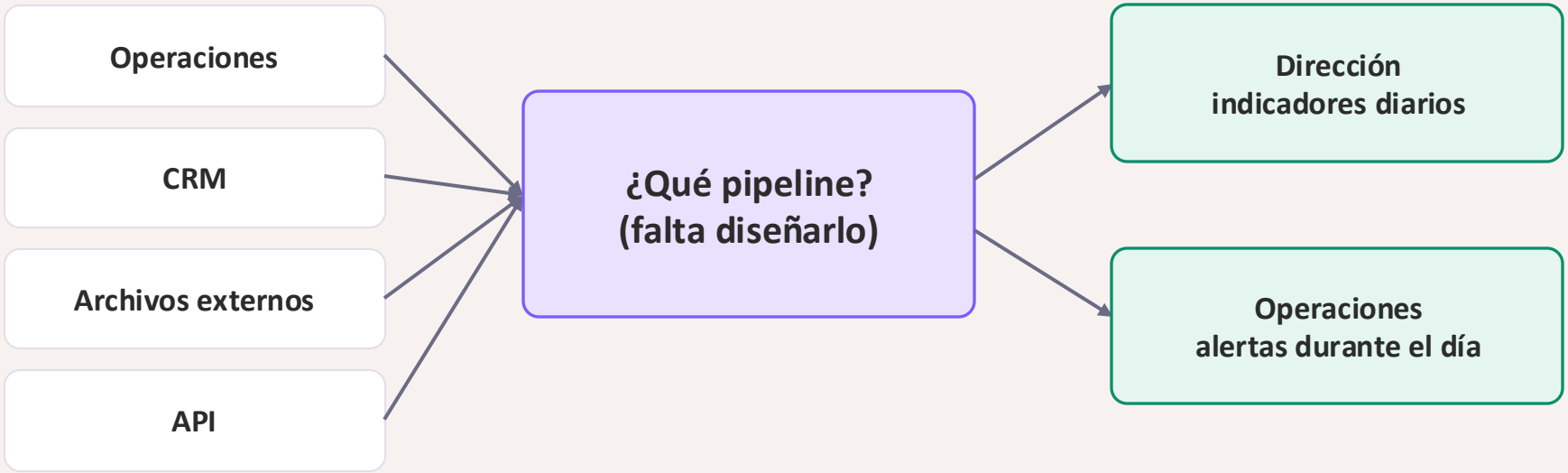
Qué decisión sería más costosa

¿Qué decisión de tu propuesta sería más costosa de revertir dentro de un año?

- Costo
- Seguridad
- Calidad
- Operación
- Dependencia tecnológica

No buscamos la arquitectura perfecta: buscamos una decisión defendible.

NexoServicios, versión incompleta



Hay información sensible

El equipo es pequeño

Hasta siete preguntas antes de implementar

1 Volumen o latencia

¿Cada cuánto se consulta y en qué ventana hay que tenerlo?

2 Seguridad

¿Qué datos son sensibles y quién puede verlos?

3 Calidad

¿Qué es «dato correcto» y cómo se mide?

4 Recuperación

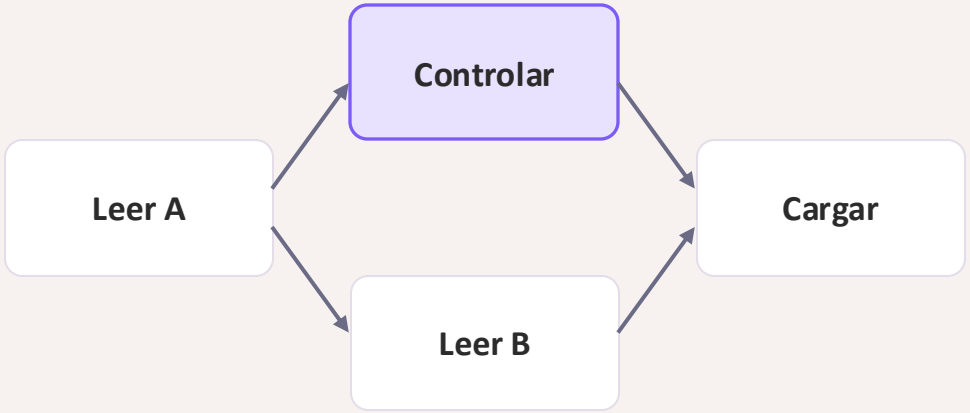
Si un día falla, ¿se reejecuta, desde dónde y quién decide?

5 Responsabilidad humana

¿Quién es dueño de cada fuente y quién responde si falla?

Una lista de productos o tecnologías todavía no alcanza.

Qué es un orquestador y una DAG



Tareas con dependencias, sin ciclos

Cuándo: Programación y horarios

En qué orden: Dependencias entre tareas

Qué hace si falla: Reintentos, avisos y reproceso

Coordina, no transforma: el cálculo pesado se hace en la base de datos.

Los siete conceptos para leer una DAG

Concepto	Qué es	En NexoServicios
DAG	El conjunto de tareas y su orden, sin ciclos	Indicadores diarios
Tarea	Una unidad de trabajo	Leer el CRM
Dependencia	Qué tiene que terminar antes	Transformar espera a los controles
Programación	Cuándo corre	Todos los días a las 6:00
Reintentos	Qué hace si falla	Dos, con espera creciente
Sensor	Espera algo, con plazo	El archivo, hasta las 7:30
Backfill	Reprocesa fechas pasadas	Los últimos siete días

Alternativas a Airflow

Herramienta	Qué es	Se define con	Conviene cuando
Apache Airflow	Orquestador de lotes de código abierto	DAGs en Python	Hay muchas fuentes y se quiere un ecosistema grande
Dagster	Orquestador centrado en los datos que produce cada paso	Python: activos y trabajos	Importan el linaje y la calidad como código
Prefect	Orquestador en Python con flujos dinámicos	Python con decoradores	Se quiere arrancar rápido con flujos que cambian
Luigi	Biblioteca de dependencias entre tareas	Clases de Python	Son cadenas de lotes simples
Argo Workflows	Orquestador nativo de Kubernetes	YAML	Todo ya corre en contenedores sobre Kubernetes
Servicios de la nube	Azure Data Factory, AWS Step Functions, Cloud Composer	Interfaz visual, JSON o DAGs	No se quiere operar la plataforma
Cron y scripts	El programador del sistema	Una línea de configuración	Una tarea simple con un solo dueño

Apache NiFi no es un orquestador: mueve datos con un flujo continuo y se complementa con uno.

Para qué sí usar un orquestador

Tareas con dependencias

Una no puede empezar hasta que otra termine.

Horarios y ventanas

Tiene que estar listo antes de una hora.

Reintentos y alertas

Las fallas transitorias se absorben y las graves avisan.

Reprocesar el pasado

Volver a correr fechas sin duplicar.

Saber qué pasó

Qué corrió, cuándo y con qué resultado.

Varias fuentes o responsables

Hay que coordinar a más de una persona o sistema.

Para qué no usarlo

Una sola tarea simple

Un cron o un script con alerta alcanza.

Respuesta en segundos

Eso es eventos o streaming; el orquestador coordina solo lo que va por lotes.

Un proceso de una sola vez

No hay nada que programar ni reprocesar.

Nadie que opere la plataforma

Un servicio administrado o scripts con alertas.

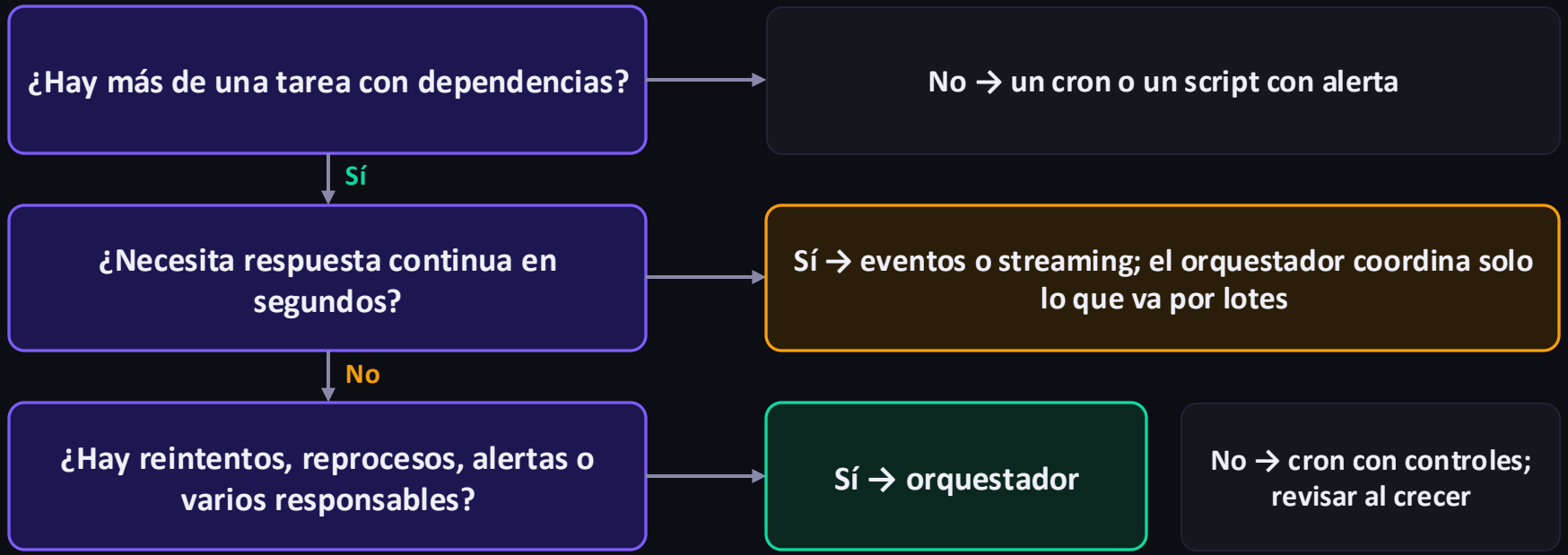
Transformaciones pesadas adentro

El cálculo va a la base de datos o al motor.

El requisito no está fijado

Primero el requisito, después la herramienta.

Un árbol para decidir



Quién responde por el dato

Dueño

Decide el uso del dato y rinde cuentas

Responsable de calidad

Cuida las reglas y la calidad

Operación

Mantiene el flujo funcionando

Consumidores

Usan el dato para decidir

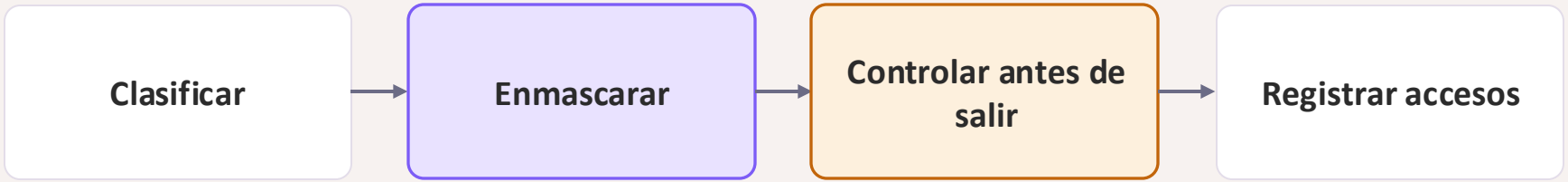
Gobierno: qué es el dato, quién responde, quién lo ve y quién lo consume.

Políticas · catálogo · clasificación · linaje · acceso

Un acuerdo escrito sobre el dato

Campo	Ejemplo: indicadores_diarios
Dueño	Dirección Comercial
Responsable de calidad	Gobierno de Datos
Columnas y clasificación	fecha y canal: públicas · órdenes e ingresos: internas
Frescura máxima	26 horas
Consumidores	Dirección y Finanzas
Controles	Sin datos sensibles expuestos · clave única

Se protegen antes de salir



Antes
DNI: 44074080

Después
DNI: 3354641e6e

El mismo valor da siempre el mismo código: se puede unir sin exponerlo.

La matriz RACI

Actividad	Arq.	Oper.	Gob.	Rev.
Definir el contrato de datos	R	C	A	I
Construir y versionar la DAG	A/R	C	I	C
Clasificar y proteger lo sensible	C	I	A/R	I
Decidir qué pasa si un control bloqueante falla	C	R	A	I
Responder alertas y recuperar	C	A/R	I	I
Aprobar un cambio de esquema	R	C	A	C

R hace · A rinde cuentas (uno solo) · C consultado · I informado

Más que «no hay nulos»

Dimensión	La pregunta	Un control de ejemplo
Completitud	¿Faltan datos clave?	Sin nulos en id_cliente y monto
Unicidad	¿Hay filas repetidas?	Clave única en id_orden
Validez	¿El valor tiene sentido?	Monto entre 1 y 50.000
Consistencia	¿Tiene la forma acordada?	Esquema esperado
Integridad	¿Existe lo que referencia?	Cada id_cliente está en el CRM
Frescura	¿Es de la fecha esperada?	El último dato es de hoy
Volumen	¿Es una cantidad razonable?	480 filas, con 30 % de tolerancia

Bloqueantes y no bloqueantes

BLOQUEANTE

Detiene todo lo que depende.

Cuando seguir daría un resultado incorrecto o inseguro.

Ejemplos: clave única, esquema, datos sensibles.

NO BLOQUEANTE

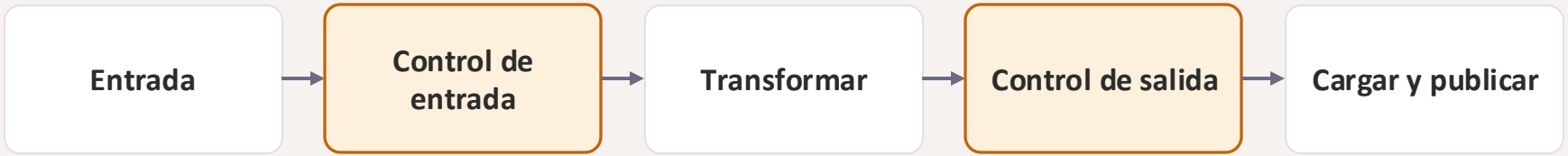
Deja una alerta y sigue.

El dato sirve, pero hay que revisarlo.

Ejemplos: montos fuera de rango, volumen.

Todo control dice qué hacer si falla y quién decide.

Dónde van los controles y cómo se recupera



Detectar
¿Cómo nos enteramos?

Contener
¿Cómo evitamos el daño?

Recuperar
¿Cómo se vuelve a correr?

Decidir
¿Quién resuelve?

Monitorear no es recuperar.

Cómo pensarlo: cinco pasos

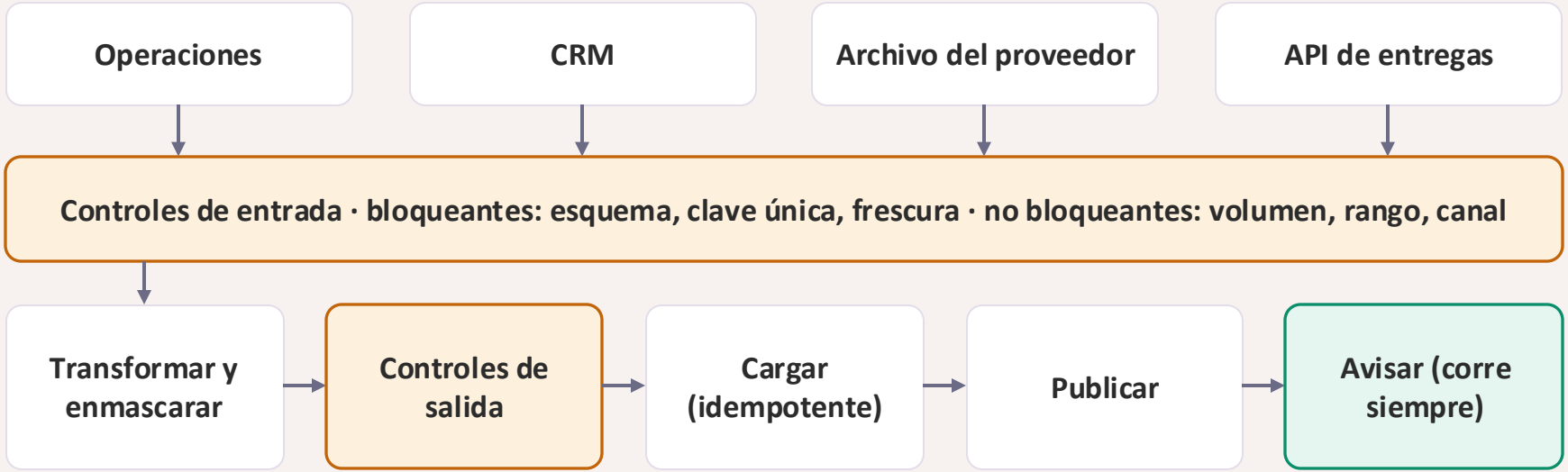


Decisión A · ¿Cada cuánto corre?
Una vez por día: 1 corrida y no 288. Costo aceptado: sin datos intradía

Decisión B · ¿Y si el archivo no llega?
Publicar y avisar: el indicador no lo usa. Si lo usara, sería bloqueante

Cada decisión declara qué se descarta y cómo se sabría que dejó de funcionar.

La DAG de NexoServicios



Lo que está en el mismo nivel puede correr en paralelo; lo demás espera.

Cuando algo falla

La API falla dos veces

3 intentos y 90 segundos de espera. La corrida es exitosa

La API se cae del todo

La tarea falla, las demás quedan omitidas y «avisar» corre. No se publica nada

El archivo no llega

Bloquear o publicar y avisar: depende de si el indicador lo usa

Monitorear dice qué pasó; recuperar define qué se hace.

La demo en Airflow

El sensor espera

Se suelta el archivo del proveedor y la DAG sigue

Reintentos

fallas_api = 2: falla, espera y se recupera

Control bloqueante

defecto = duplicados: se frena y no se publica

Control no bloqueante

defecto = montos_raros: alerta y sigue

Datos sensibles
defecto = pii: el contrato frena la salida

Esquema cambiado
./soltar_archivo.sh errores

Reproceso
Volver a correr el mismo día: siempre 3 filas

Es la misma DAG de la clase, en Airflow real con Docker.

Ocho buenas prácticas de pipelines

Idempotencia

Correr dos veces deja el mismo resultado.

Tareas chicas

Cada una con un solo propósito.

La fecha de la corrida

No la hora actual: así se puede reprocesar.

Secretos afuera

Nunca claves en el código.

Reintentos acotados

Con espera creciente y un tope.

Alertas con dueño

Dicen qué hacer y quién decide.

Dejar registro

Qué corrió, cuándo y con qué resultado.

Probar con fallas

Simular antes de producción.

Seis errores que se repiten

Error	Qué pasa	Cómo se evita
Cargar sin idempotencia	Un reproceso duplica: ingresos de 848.651 a 1.697.302	Reemplazar por clave
Frecuencia sin requisito	Cada 5 minutos son 288 corridas contra 1	Fijar la latencia primero
Control sin acción	Detecta, pero nadie sabe qué hacer	Acción y responsable en cada control
Reintentos sin límite	Se agota la cuota de la fuente	Acotar y esperar más cada vez
Hora actual en la tarea	El reproceso da otro resultado	Usar la fecha de la corrida
Tablas por el mensaje entre tareas	Se llena la base de metadatos	Pasar rutas y conteos

El desafío: cómo se trabaja

Arquitectura

Diseña la DAG

Operación

Piensa cómo se opera y se recupera

Gobierno

Cuida el contrato, lo sensible y los responsables

Revisión

Pregunta por evidencia

Una DAG comentada

Una matriz RACI breve

Controles bloqueantes y no bloqueantes

Cada decisión incluye una alternativa que se descartó.

Siete equipos, siete desafíos

1 Indicadores diarios

Dependencias y qué se publica si una fuente llega tarde.

2 Alertas durante el día

La frecuencia sale del requisito de latencia.

3 Archivo tarde o repetido

Sensor, versión del archivo e idempotencia.

4 API con límite y caídas

Reintentos, tandas y reproceso sin saturar.

5 Datos sensibles

Clasificar, enmascarar y registrar accesos.

6 El esquema cambió

Contrato, esquema bloqueante y recuperación.

7 ¿Hace falta un orquestador?

Decidir con criterio: sí o no.

Seis preguntas que tiene que responder

1. ¿Qué problema y qué decisión resuelve?
2. ¿Qué entra y qué queda fuera de alcance?
3. ¿Qué alternativa se descartó y por qué?
4. ¿Cómo se prueba antes de pasar a producción?
5. ¿Cómo se detecta, se contiene y se recupera una falla?
6. ¿Quién decide cuando un control no pasa?

Están al final del notebook, en la ficha de entrega.

Revisión cruzada con una lista

- ☐ El problema está formulado como decisión y no como deseo
- ☐ Las fuentes, transformaciones y consumidores son identificables
- ☐ Los límites de seguridad y responsabilidad están visibles
- ☐ Existe un criterio de calidad medible
- ☐ Hay una estrategia de reintento, reproceso o rollback
- ☐ Se registra evidencia suficiente para explicar qué ocurrió
- ☐ El costo o complejidad aceptados están declarados

Preguntas para el debrief

1. ¿Qué decisión cambiaría si la latencia requerida se redujera diez veces?
2. ¿Qué componente concentra el mayor radio de impacto ante una falla?
3. ¿Qué evidencia necesitaría una persona ajena para reconstruir lo ocurrido?
4. ¿Qué control podría generar falsos positivos y cómo se administraría?
5. ¿Qué parte del diseño depende de una política o infraestructura todavía no confirmada?

Qué me llevo y qué dejo escrito

CUATRO IDEAS PARA LLEVARSE

1. **Cinco pasos para razonar**
   Restricción, opción, trade-off, control y evidencia.
2. **El requisito manda**
   Un orquestador se decide con el requisito, no con la moda.
3. **Calidad es acción**
   Cada control: bloqueante o no, con responsable.
4. **Monitorear no es recuperar**
   Detectar, contener, recuperar y decidir.

TICKET DE SALIDA · UNA FRASE POR CADA UNO

Mi decisión más sólida

La que mejor puedo defender.

Un riesgo abierto

Lo que todavía no controlo.

El próximo experimento

Qué probaría después.

LO QUE VIENE

Hacia la clase 5

Hoy: pipelines de datos con calidad y gobierno → **Clase 5: pipelines de modelos y plataformas**

- **Dependencias**
- **Controles**
- **Responsables**
- **Recuperación**

Las mismas ideas, con un modelo en lugar de una tabla.

DIGITALHOUSE

ARCA | AGENCIA DE RECAUDACIÓN Y CONTROL ADUANERO

PREGUNTAS Y RESPUESTAS

Q&A

¿Qué dudas, casos o ideas quedaron?

¡Muchas gracias!

Laura Sol Franco · Digital House

Programa: Arquitectura de Datos, Gobierno y Orquestación — ARCA